

Correct Optimizations: Phase 1 Report

Based on Minotaur: A SIMD-Oriented Synthesizing Superoptimizer [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

Claudia Dünzinger
Technical University Munich

1 Introduction

This report aims to analyze and propose opportunities to continue the research from Liu et al. [1].

2 Paper Summary

2.1 Context

Minotaur is a superoptimizer that tries to identify missed optimization opportunities by the LLVM compiler by slicing generated IR and trying to synthesize new faster code. Its synthesizer creates improved machine code, generating and comparing equivalent but cheaper instruction sequences. To guarantee correctness of the optimization, Alive2 was extended to formally verify the generated code. Additionally, to reduce computation overhead, Minotaur also caches discovered transformations for reuse them in future compilations.

The use case of Minotaur is to further optimize already heavily optimized code from a compiler which still often contain suboptimal code segments, particularly for SIMD operations. The authors showed that even LLVM’s advanced autovectorizer produced redundant or inefficient SIMD instructions. For those missed opportunities, Minotaur can correct these suboptimal instruction sequences by replacing them with improved machine code during the compilation. Some of those optimizations could be generalized and added to LLVM. Hence, the authors enabled higher performing and additionally formally verified machine code generation through the use of Minotaur.

Before Minotaur, several superoptimizer already existed that search through the code, be it assembly or LLVM IR, for more efficient sequences instead of performing set rewriting rules like traditional compilers. Common examples include Souper – with focus on scalar integer optimizations and which also uses LLVM IR – and STOKE, which uses stochastic search for faster x86 code blocks. However, those tools only addressed part of the problem. Souper [2] detects only improvements for integer operations, but not for floating-point or

SIMD instructions. STOKE’s [3] improvements include complex optimizations, but the tool relies on randomized testing and not formal verifications for its transformations. Therefore, the combined challenge of performing complex vector or floating point optimizations, formally verifying the transformation and integrating those capabilities into production compilers remained unsolved, until Minotaur.

Designing a efficient and high performing superoptimizer included several challenges. First, the search space of replacement options for a suboptimal code sequence is enormous, making an exhaustive search for the optimal choice computationally expensive. Second, the semantic formal and not random equivalency check adds further overhead – especially for hard to verify SIMD and floating point operations. Minotaur relies on SMT solvers for equivalency checks between the original code and the proposed optimization, but these solvers often struggle with large complex expressions. At the same time a sufficiently large number of lines of code is needed to provide enough context to be able to find improvement options in the first place. Furthermore, since Minotaur aims for low level, hardware aware optimizations, the authors used detailed information about pipelines and cache effects to accurately model the performance of optimizations. These difficulties of realizing high performance without too much computational cost, required a multiple stages process.

2.2 Contributions

Because today’s SMT solvers cannot optimize entire LLVM functions, Minotaur instead optimizes each instruction individually. It therefore breaks the LLVM function in SSA form down into cuts, which are small loop-free subsets of the the instruction’s dependencies, to capture where the data used by the instruction came from. The size of the cut hereby influences the optimization, as too small cuts contain not enough context to determine a better code sequence, while too large cuts increase the solvers computation time dramatically. Minotaur’s cut extraction is noticeably better than previous ones since it is able to handle memory operations, vector op-

erations and floating-point operations. Through an empirical study the authors selected the cut size of four as it provides a good balance between finding most optimization while still running in an acceptable amount of time. For each cut, a set of potential alternatives which are faster are synthesized. These improvements are then verified with Alive2, which the authors extended to support SIMD operations. Lastly, for each optimization its speedup is estimated using a more accurate model (LLVM MCA) and its original and transformed code is cached for reuse in future compilations.

To measure how Minotaur’s optimizations influence the overall performance of larger software projects, the authors benchmarked Minotaur on the SPEC CPU 2017, GMP and libYUV on Intel and AMD hardware with LLVM 18 compiled with -O3 and -march=native. They determined a maximum speedup of 1 - 7 percent, where integer SIMD fragments made up 75 percent of the discovered rewrites. When the compilation is performed with an empty cache, Minotaur increased compile times significantly. With a filled cache, however, only an additional 3 percent of computation time was needed. Overall, the authors found 15 simple and general floating-point and vector optimizations, which they then reported to the LLVM project.

3 Artifact Evaluation Results

First, it has to be mentioned that the paper did not provide a exact git commit hash for the minotaur version that was used by the authors to run their benchmarks. Since we could not determine which version was used it was decided to evaluate the most up to date version of minotaur¹. One important difference between the most recent version and the version referenced by the authors in the paper is that the version in the paper relied on LLVM 18 whereas the newest version uses LLVM 20.

Second, the most up to date version of minotaur, as of writing, does not work out of the box. For it to not immediately crash on startup, the order in which the libraries are linked by minotaur had to be changed. This changes the order in which the offending dynamic libraries are loaded when the program is started and thereby fixing the issue of undefined symbols.

Third, minotaur uses alive2’s LLVM interpreter which is listed as highly experimental² and caused multiple crashes. Some of which were not reliably reproducible and could therefore not be debugged. In order to mitigate this, the compiler was run multiple times or problematic source files were filtered. More about that when the setup for libyuv is discussed. Some other bugs could be fixed.

¹<https://github.com/KiSchulz/minotaur/tree/8ae5395286ba40fa29c16a0229fd3d6e9795d4cd>

²<https://github.com/AliveToolkit/alive2/tree/master?tab=readme-ov-file#running-the-standalone-llvm-execution-tool-alive-exec>

Fourth, the caching of minotaur seems to not fully work in the last release, as compiling a source file multiple consecutive times still leads to cache misses. This might be caused differing inputs from LLVM’s side. Nevertheless it produces different binaries than running LLVM without the minotaur plugin.

After those issues were addressed, as far as possible, a run could be attempted. Everything was compiled with the -O3 and -march=native flags as described in the paper. We benchmarked libgmp-6.2.1, as the paper did, and a fitting version of libyuv, which is able to build with LLVM 20. All runs were performed with the same parameters as the paper described i.e. a depth bound of 4, total synthesizes time of 5 minutes and a 1 minute timeout for individual smt queries. The benchmarks were build and run on an AMD Ryzen 9 5900X whereas the authors used an AMD Ryzen 9 5950X. The former has 12 cores running at up to 4.8 GHz whereas the latter has 16 cores running at up to 4.9 GHz.

libgmp: According to the paper the build with an empty cache took 440 minutes for libgmp. Our build took approximately 1020 minutes while using a partly dirty cache from previous attempts. It seems unlikely that the difference in build times are caused by the hardware. This difference in compile time could be caused by the authors using the offline mode of minotaur which we intentionally did not use since as discussed earlier the cache did not work as expected.

After finishing the build of libgmp the provided tests were run. When building with vanilla LLVM all tests passed without any issue. In contrast, using minotaur at least two tests failed namely t-root and t-perfpow. This indicates some correctness bug in alive2 or in how minotaur uses alive2. Later more of that is discussed.

Ignoring the failing tests and the other issues, we could not see any speedup when running the benchmark using minotaur 1. There could be multiple reasons for this. One could be that any of the previous issues which we were not able to solve yet caused this. Or, alternatively that most of the relevant optimization found by minotaur in LLVM 18 are now already present in LLVM 20. This is not entirely implausible since the authors were able to get some of the found optimizations merged into LLVM. The high variance between the single benchmarks could either stem from minotaur optimizations or be caused by the system executing the benchmark. Nevertheless the overall results did not change significantly between the benchmark runs.

libyuv: In order to be able to even build libyuv and to not run into constant crashes during the compilation phase, its third party dependencies were compiled without minotaur. This would probably not have influenced the results to much since those dependencies seemed to be used mainly for auxiliary functionality. Due to us not compiling the third party dependencies comparing compile times makes little sense since the authors seemed to compile all sources using minotaur judging by their compile time. But in the end we were not

able to run the benchmark suite of libyuv, which doubles as its test suite, since it was failing the tests using minotaur. Similar to the results for libgmp the vanilla LLVM build passed the tests with no issues.

These results prompted us to investigate further. We decided to try to reproduce some of the optimizations for short LLVM IR code snippets listed in the paper.

After those failed as well we decided to write our own very simple program to provoke errors.

4 Research Ideas

TODO

- Make the caching work

4.1 Dynamic Cutting Depth using Profile Guided Optimization

One major down side of minotaur at the moment is its long compile times. Even for small projects, which usually compile within seconds, it can take thousands of times longer, making it hours or even days. Improving on compile time could enable a more practical use of minotaur. The main challenge is how to save on time while compiling and to not lose the benefits of minotaur i.e. runtime performance improvements.

To achieve this one could try, as the paper suggested, to dynamically set the cutting depth. Determining the cutting depth for each cut is hard since one would have to know how important a piece of code is to the overall runtime performance. One simple way of doing this would be to first build the given code without minotaur and then benchmark it while gathering data with a profiler. This profiling data could then be used in a second compilation of the code using minotaur to determine the cutting depth in minotaur dynamically during compilation. In theory two goals could be achieved, namely reducing total compile time, by not spending time on unimportant code like code that is only ever executed once, and second, it could also enable minotaur to spend more time on important code which actually impacts performance. Changes in compile time between the papers version of minotaur and the augmented version would be the first important metric. But this is obviously not enough, one also has to measure the runtime in order to determine if the proposal delivered any benefits. Ideally the suggestion would improve compile times by not optimizing non critical code while keeping or maybe improving runtime of the code.

- try to optimize how minotaur and alive and z3 interact
- improve code generation
- only use the complex runtime estimate (llvm mca) model
- to do less m3 work
- add fast math support
- multi pass the superoptimizer

References

- [1] Zhengyang Liu, Stefan Mada, and John Regehr. Minotaur: A SIMD-oriented synthesizing superoptimizer. *Proceedings of the ACM on Programming Languages (OOPSLA2)*, 8(OOPSLA2):1–25, October 2024. doi: 10.1145/3689766. <https://doi.org/10.1145/3689766>.
- [2] Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Jubi Taneja, and John Regehr. Souper: A synthesizing superoptimizer. *CoRR*, abs/1711.04422, 2017. URL <http://arxiv.org/abs/1711.04422>.
- [3] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. *CoRR*, abs/1211.0557, 2012. URL <http://arxiv.org/abs/1211.0557>.

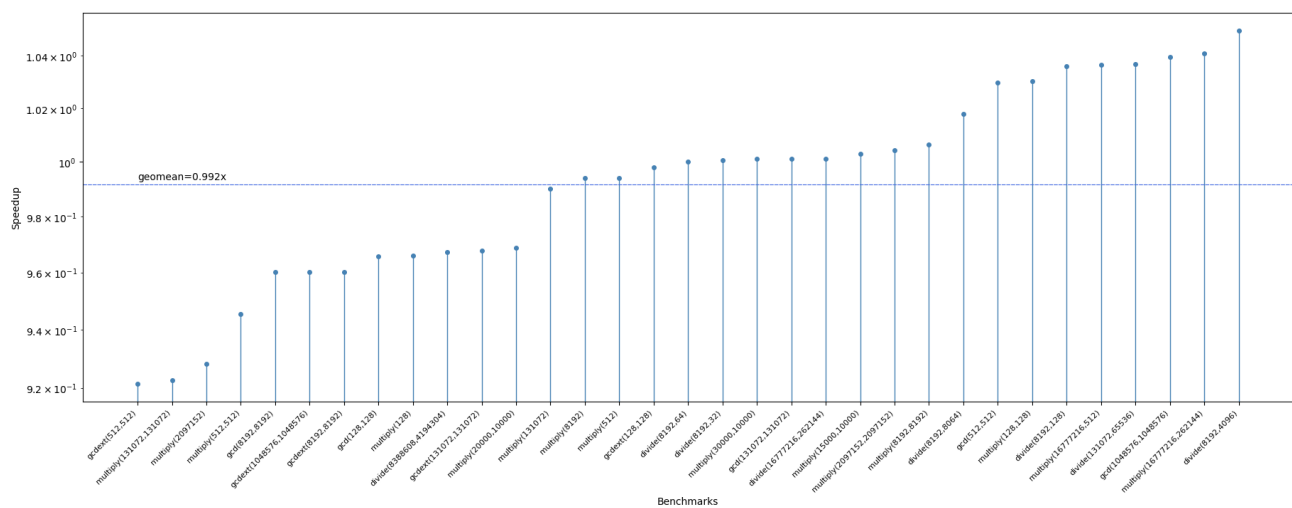


Figure 1: Our libgmp benchmark results